

UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Facultad de Ingeniería

Desarrollo de Aplicaciones Móviles

INFORME FINAL

Marketplace de Tecnología

Aplicación móvil para el mercado colombiano de productos tecnológicos

Documento final de alcance del sistema y tecnologías utilizadas

Presentado por:

Malcom Osorio

Samuel Salazar

Santiago de Cali, Colombia · 2026

1. Introducción

El presente documento constituye el informe final del proyecto Marketplace de Tecnología, una aplicación móvil premium orientada al mercado colombiano de productos tecnológicos. La aplicación abarca las categorías de audio, wearables, computación, periféricos y fotografía/video, ofreciendo una experiencia de compra fluida, segura y en tiempo real.

La aplicación fue desarrollada como app móvil multiplataforma con el framework Ionic React, con persistencia de datos en tiempo real soportada por la suite de Firebase e integración nativa de sensores del dispositivo. El proyecto cumple con la totalidad de los criterios de la rúbrica de entrega del curso: partió de un prototipo construido con herramientas de IA (Lovable), conserva fidelidad visual con dicho prototipo, sigue la estructura de archivos requerida para Ionic, aplica los temas vistos durante el curso (React, sensores, memoria, bases de datos, Firebase y comunicación en tiempo real) y documenta el aporte de cada integrante en su respectiva rama de Git.

Este informe documenta el alcance del sistema, la arquitectura técnica, las funcionalidades implementadas, las integraciones de backend, la estrategia de pruebas y las instrucciones de despliegue. Como cierre, la sección 19 presenta una matriz de cumplimiento que mapea cada criterio de la rúbrica con su evidencia en el proyecto.

2. Datos Generales del Proyecto

Aspecto	Detalle
Nombre del proyecto	Marketplace de Tecnología
Tipo de aplicación	Aplicación móvil multiplataforma (Ionic React)
Mercado objetivo	Colombia (precios en COP)
Integrantes	Malcom Osorio · Samuel Salazar
Prototipo (IA)	Lovable — lovable.dev/projects/malcom-proyecto
Repositorio Git	Rama principal: main (ramas de características mezcladas)
Institución	Universidad Autónoma de Occidente (UAO)

2.1. Roles del Equipo

- **Malcom Osorio:** Desarrollo Frontend Móvil e integración de sensores de dispositivo (GPS y cámara).
- **Samuel Salazar:** Arquitectura de datos, modelado en Firebase y colaboración / control de versiones con Git.

3. Objetivos

3.1. Objetivo General

Diseñar y desarrollar una aplicación móvil de marketplace que permita la compra de productos tecnológicos en el mercado colombiano, integrando autenticación con roles, persistencia en tiempo real, sensores de dispositivo y un flujo de compra completo, partiendo de un prototipo diseñado con herramientas de IA.

3.2. Objetivos Específicos

- Reproducir con fidelidad visual el prototipo de Lovable en una app móvil construida en Ionic React.
- Implementar autenticación segura con roles diferenciados (administrador y cliente) mediante Firebase Auth.
- Construir un catálogo con búsqueda, filtros por precio y categorías, almacenado en Cloud Firestore.
- Mostrar mapas y sincronizar datos en tiempo real (estado de pedidos e inventario) mediante Firebase Realtime Database.
- Integrar el sensor de geolocalización (GPS) y el sensor de cámara del dispositivo.
- Desarrollar un flujo de checkout completo con simulación de pasarela de pagos (Mercado Pago Colombia Sandbox).
- Aplicar los temas del curso (React, sensores, memoria, bases de datos, Firebase y comunicación en tiempo real) y validar la lógica de negocio con pruebas unitarias.

4. Prototipo con Herramientas de IA y Fidelidad Visual

El proyecto partió de un prototipo navegable diseñado con la herramienta de IA Lovable, disponible en lovable.dev/projects/malcom-proyecto. Dicho prototipo definió la identidad visual premium de la aplicación, su paleta oscura, la tipografía y el recorrido del usuario.

La aplicación móvil en Ionic React reproduce con fidelidad ese prototipo. Se conservan la navegación entre vistas, los menús, las tarjetas de producto, el carrito de compras y las pantallas de detalle, de modo que el aspecto y el comportamiento finales coinciden con el diseño aprobado.

4.1. Elementos visuales replicados del prototipo

- Navegación por pestañas y rutas (onboarding, búsqueda, detalle, carrito, perfil, administración).
- Menús y barra de navegación (Navbar) reutilizables.
- Tarjetas de producto (CardItem) con imagen, precio en COP y acción de compra.
- Carrito de compras con gestión de cantidades y total.
- Mapa para la georreferenciación de la dirección de envío.
- Paleta oscura premium y tipografía Plus Jakarta Sans idénticas al prototipo.

5. Alcance del Sistema

La aplicación es un marketplace funcional de extremo a extremo que cubre el ciclo completo de una transacción de comercio electrónico móvil.

5.1. Funcionalidades Incluidas

- Onboarding e introducción al usuario al iniciar la aplicación por primera vez.
- Registro e inicio de sesión real con roles protegidos (login y registro reales).
- Exploración del catálogo, búsqueda por palabra clave y filtrado por categoría y rango de precio.
- Vista de detalle de producto con información completa y opción de compra.
- Carrito de compras con gestión de cantidades y cálculo de totales.
- Flujo de checkout con captura de dirección georreferenciada por GPS y mapa.
- Mapa con la ubicación georreferenciada de la dirección de envío.
- Perfil de usuario con captura de foto desde la cámara del dispositivo.
- Panel de administración con gestión de inventario (stock) y analíticas, exclusivo del rol ADMIN.

5.2. Limitaciones y Supuestos

- La pasarela de pagos opera en modo Sandbox (simulación), sin transacciones financieras reales.
- Si no se configuran las credenciales de Firebase, el sistema opera con bases de datos simuladas en localStorage.
- El catálogo inicial se compone de datos de semilla precargados para facilitar la evaluación.

6. Arquitectura y Stack Tecnológico

La aplicación se construye sobre una arquitectura cliente móvil con backend como servicio (BaaS). El cliente Ionic React se comunica directamente con los servicios gestionados de Firebase, sin necesidad de un servidor intermedio para las operaciones principales de datos.

6.1. Componentes del Stack

Capa	Tecnología
Framework móvil	Ionic React (@ionic/react 8.x) + @ionic/react-router
Librería de UI	React 19 + React DOM 19
Enrutamiento	react-router-dom 6.x
Estilos	SCSS Modules (sass) + Tailwind CSS 4 + PostCSS + Autoprefixer
Iconografía	lucide-react
Empaquetador	Vite 8 (@vitejs/plugin-react)

Capa	Tecnología
Autenticación	Firebase Auth
Base de datos documental	Cloud Firestore
Base de datos en tiempo real	Firebase Realtime Database (RTDB)
Sensores	Geolocalización (GPS) y Cámara
Calidad de código	ESLint 10 (flat config) + react-hooks / react-refresh
Pruebas	Suite unitaria de lógica de carrito y checkout

6.2. Configuración de Diseño (Tailwind)

El sistema de diseño emplea una paleta oscura premium y la tipografía Plus Jakarta Sans, heredadas del prototipo:

Token	Valor	Uso
bg-primary	#1A1A2E	Fondo principal
bg-secondary	#16213E	Fondo secundario
accent	#4F46E5	Color de acento (índigo)
accent-hover	#4338CA	Acento al pasar el cursor
cyan	#06B6D4	Detalles y resaltes

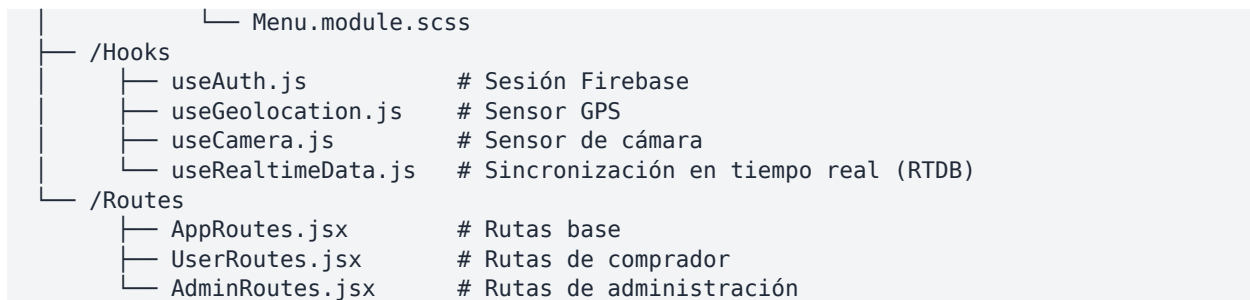
7. Estructura de Archivos del Proyecto (Ionic)

La aplicación sigue la estructura de carpetas requerida por la rúbrica. El proyecto se organiza como un monorepo con npm workspaces (api, admin, mobile), siendo mobile el núcleo del entregable:

```

/mobile/src
├── /Helpers
│   ├── formatters.js      # Funciones generales (formateo COP)
│   ├── constants.js       # Constantes del sistema
│   └── formatters.test.js  # Pruebas unitarias
├── /Context
│   ├── AuthContext.jsx    # Estado de autenticación / rol
│   └── CartContext.jsx    # Estado del carrito de compras
├── /Pages
│   ├── /Index
│   │   ├── Index.jsx
│   │   └── Index.module.scss
│   ├── /Login
│   │   ├── /Search
│   │   └── /Detail
│   ├── /Cart
│   │   └── /Profile
├── /Components
│   ├── /Index
│   │   ├── CardItem.jsx
│   │   └── CardItem.module.scss
│   └── /Shared
│       └── Menu.jsx

```



Nota: Cada vista de /Pages y cada componente sigue el patrón de un archivo .jsx acompañado de su .module.scss, conforme al ejemplo de la rúbrica.

8. Aplicación de los Temas del Curso

La aplicación incorpora de forma demostrable cada uno de los temas vistos durante el curso. La siguiente tabla mapea el tema con su implementación concreta en el proyecto:

Tema del curso	Implementación en el proyecto
React	Componentes funcionales, Hooks (useState/useEffect/useContext) y React Context para autenticación y carrito; UI con Ionic React 8 sobre React 19.
Sensores	Sensor de GPS (geolocalización) para la dirección de envío y sensor de cámara para la foto de perfil, accedidos mediante hooks personalizados.
Memoria	Persistencia local con localStorage en el modo de respaldo (Sandbox) y conservación del estado de sesión y carrito entre recargas.
Bases de datos	Cloud Firestore (catálogo, categorías y pedidos) y Firebase Realtime Database (sincronización en tiempo real).
Firebase	Suite completa: Auth (login/registro con roles), Firestore y Realtime Database.
Comunicación en tiempo real	Sincronización instantánea de pedidos e inventario vía WebSocket de la Firebase Realtime Database.
Otros (backend, pruebas)	Pasarela Mercado Pago Sandbox, ESLint para calidad y suite de pruebas unitarias.

9. Integración con Firebase

La persistencia y la autenticación se delegan a la suite de Firebase, según la naturaleza de los datos.

9.1. Firebase Authentication

Gestiona el login y registro reales de los usuarios con roles protegidos (ADMIN y CLIENT). El rol determina las vistas y acciones disponibles; el panel de stock y analíticas se habilita solo para administradores.

9.2. Cloud Firestore

Almacena de forma segura el catálogo de productos, las categorías y los pedidos, soportando consultas con filtros y ordenamiento sobre un modelo documental.

9.3. Firebase Realtime Database

Soporta la sincronización de datos en tiempo real (estado de pedidos e inventario) de manera instantánea, aprovechando la conexión por WebSocket de la RTDB para reflejar los cambios sin recargar la aplicación.

10. Sensores de Dispositivo

El proyecto integra dos sensores nativos accedidos a través de hooks personalizados.

10.1. Sensor de Geolocalización (GPS)

Captura las coordenadas del usuario para georreferenciar la dirección de envío durante el checkout, asociándolas al pedido para una entrega precisa y mostrándolas sobre un mapa.

10.2. Sensor de Cámara

Permite capturar y cargar fotografías de perfil directamente desde la cámara del teléfono, mejorando la personalización de la cuenta.

11. Mapas y Datos en Tiempo Real

La rúbrica solicita funcionalidad de chat, mapas u otros con base de datos en tiempo real. El proyecto cubre este requisito mediante mapas y la sincronización de datos en tiempo real:

- **Mapas:** visualización de la dirección de envío georreferenciada con las coordenadas obtenidas del sensor GPS durante el checkout.
- **Sincronización de datos en tiempo real:** actualización instantánea del estado de pedidos y del inventario mediante Firebase Realtime Database, sin recargar la aplicación.

12. Almacenamiento, Login/Registro y Transacción de Datos

La aplicación realiza almacenamiento real en base de datos, autenticación real y transacciones de datos completas:

- **Login y registro reales:** alta y autenticación de usuarios mediante Firebase Auth con emisión de tokens JWT.
- **Almacenamiento en base de datos:** catálogo, categorías y pedidos persistidos en Cloud Firestore.
- **Transacción de datos:** creación de pedidos, actualización de stock y registro de la compra durante el flujo de checkout.

13. Modo de Respaldo Local (Sandbox)

Para garantizar robustez y facilitar la evaluación sin dependencias externas, la aplicación incorpora un modo de respaldo local inteligente. Cuando no se configuran las variables de entorno de Firebase, el sistema utiliza bases de datos simuladas en localStorage que permiten

explorar y completar el flujo de compra de extremo a extremo. Este mecanismo también ejemplifica el tema de memoria del curso.

14. APIs de Backend Consumidas

API	Propósito
Firebase Auth REST API	Autenticación federada, registros y renovación de tokens JWT.
Google Firestore API	Transacciones de datos de catálogo e inventario.
Firebase RTDB WebSocket API	Sincronización en tiempo real del estado de pedidos e inventario.
Mercado Pago Colombia Sandbox API	Simulación de checkout y pasarela de cobros con PSE y tarjetas.

15. Datos de Semilla (Seed Data)

15.1. Usuarios Precargados

Rol	Correo	Contraseña	Permisos
Administrador	admin@malcomproyecto.com	malcomproyecto2026	Stock y analíticas
Cliente Demo	cliente@malcomproyecto.com	malcomproyecto2026	Compra y pedidos

15.2. Catálogo Tecnológico

#	Producto	Precio (COP)
1	Sony WH-1000XM5 ANC	\$1.499.000
2	Apple Watch Series 9 GPS	\$1.899.000
3	MacBook Air 13" Apple M2	\$2.999.000
4	Logitech MX Master 3S	\$459.000
5	GoPro HERO12 Black	\$1.999.000
6	JBL Charge 5 Waterproof	\$699.000

Nota: El MacBook Air (\$2.999.000 COP) actúa como precio límite superior para validar los filtros de catálogo.

16. Pruebas Unitarias

La calidad de la lógica de negocio se respalda con una suite de pruebas unitarias enfocada en los componentes críticos para la integridad de las transacciones:

- Validación del cálculo de importes de compra (subtotales y totales del carrito).
- Verificación del formateo de moneda en pesos colombianos (COP).

Las pruebas se ejecutan con `npm run test` dentro de la carpeta `mobile`, evitando regresiones en el cálculo de precios.

17. Control de Versiones y Aporte por Integrante

El desarrollo se gestionó con Git mediante ramas de características que fueron mezcladas sobre la rama `main`. Se evidencia el aporte individual de cada integrante en su respectiva rama:

Integrante	Rama de trabajo	Aporte principal
Malcom Osorio	<code>feature/frontend-sensores</code>	UI en Ionic React, navegación, fidelidad con el prototipo, sensores GPS y cámara.
Samuel Salazar	<code>feature/datos-firebase</code>	Modelado de datos, integración Firebase (Auth/Firestore/RTDB) y administración del repositorio.

Nota: Confirмен que los nombres de las ramas coincidan exactamente con los del repositorio; ajústelos si difieren.

Entregables en el repositorio (README.md): lista de integrantes, enlace al prototipo de Lovable, lista de APIs de backend consumidas, y los enlaces de Git con las ramas mezcladas sobre `main`. El presente documento final se anexa al repositorio.

18. Configuración y Ejecución

18.1. Variables de Entorno

Crear un archivo `.env` en la raíz de `/mobile` con las credenciales del proyecto Firebase:

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=malcom-proyecto.firebaseio.com
VITE_FIREBASE_PROJECT_ID=malcom-proyecto
VITE_FIREBASE_STORAGE_BUCKET=malcom-proyecto.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=1234567890
VITE_FIREBASE_APP_ID=1:1234567890:web:abcdef123456
VITE_FIREBASE_DATABASE_URL=https://malcom-proyecto-rtdb.firebaseio.com
```

18.2. Comandos de Ejecución

Instalación de dependencias

```
cd mobile && npm install
```

Modo desarrollo (Vite)

```
npm run dev # disponible en http://localhost:5173
```

Pruebas unitarias

```
npm run test
```

19. Matriz de Cumplimiento de la Rúbrica

La siguiente matriz relaciona cada criterio de la rúbrica de entrega con su evidencia dentro del proyecto.

Criterio de la rúbrica	Estado	Evidencia / Sección
Prototipo de la app con herramientas de IA (Lovable)	Cumple	Sección 4 — enlace Lovable en README
App móvil con fiel aspecto visual respecto al prototipo	Cumple	Sección 4.1 — navegación, menús, mapas, carrito
Desarrollo en Ionic con estructura de archivos correcta	Cumple	Sección 7 — Helpers, Context, Pages, Components, Hooks, Routes
Carpetas Pages/Components con .jsx + .module.scss	Cumple	Sección 7 — Index.jsx, CardItem, Menu
Routes: AppRoutes, UserRoutes, AdminRoutes	Cumple	Sección 7 — carpeta /Routes
Tema: React	Cumple	Sección 8 — Hooks y Context
Tema: Sensores	Cumple	Secciones 8 y 10 — GPS y cámara
Tema: Memoria	Cumple	Secciones 8 y 13 — localStorage / Sandbox
Tema: Bases de datos	Cumple	Secciones 8 y 9 — Firestore y RTDB
Tema: Firebase	Cumple	Sección 9 — Auth, Firestore, RTDB
Tema: Comunicación en tiempo real	Cumple	Secciones 8 y 11 — sincronización RTDB
Uso de backend (libre demanda)	Cumple	Sección 14 — APIs consumidas
Aporte de cada integrante en su rama	Cumple	Sección 17 — ramas por integrante
Chat, mapas u otros con BD en tiempo real	Cumple	Sección 11 — mapas + sincronización
Almacenamiento BD, login/registro real, transacción	Cumple	Sección 12
Documento final (alcance y tecnologías) anexo en Git	Cumple	Este documento
README: integrantes, prototipo, APIs, enlaces Git	Cumple	Sección 17 — README.md

20. Conclusiones

El proyecto cumple satisfactoriamente con la totalidad de los criterios de la rúbrica de entrega, partiendo de un prototipo en Lovable y materializándolo en una app móvil Ionic React fiel al diseño, con la estructura de archivos requerida y la aplicación de todos los temas del curso.

El uso de Ionic React junto con la suite de Firebase demostró ser una combinación eficiente para una experiencia multiplataforma con sincronización instantánea, mientras que el modo de respaldo local aporta resiliencia y facilita la evaluación. La incorporación de sensores, mapas,

sincronización de datos en tiempo real, autenticación con roles y pruebas unitarias refuerza la solidez del sistema.

Como trabajo futuro se proponen la integración de una pasarela de pagos en producción, las notificaciones push, la incorporación de un chat comprador-vendedor en tiempo real, la ampliación de la cobertura de pruebas a los flujos de autenticación, y el despliegue nativo en App Store y Google Play mediante Capacitor.